Supporting Information: ripopt — An Interior Point Optimizer in Rust with Implicit Slacks and Robust Recovery

John Kitchin

April 25, 2026

Contents

1 Introduction **4**

2 Implicit Slack Formulation **4**
2.1 Algorithm Description . . . 4
2.2 Source Code References . . . 5
2.3 Condensed KKT via Schur Complement . . . 5

3 Convergence Test **5**
3.1 Algorithm Description . . . 5
3.1.1 Dual Scaling s_d . . . 6
3.1.2 Two-Gate Test . . . 6
3.2 Source Code References . . . 6

4 Two-Phase Restoration with Multi-Attempt Recovery **6**
4.1 Algorithm Description . . . 6
4.1.1 Phase 1: Gauss-Newton Feasibility Solver . . . 6
4.1.2 Phase 2: NLP Restoration . . . 7
4.1.3 Multi-Attempt Recovery . . . 7
4.1.4 Post-Restoration State Reset . . . 7
4.2 Source Code References . . . 7

5 Automatic NE-to-LS Reformulation **8**
5.1 Algorithm Description . . . 8
5.1.1 Detection . . . 8
5.1.2 Reformulation . . . 8
5.1.3 Full Hessian . . . 9
5.1.4 Fallback . . . 9
5.2 Source Code References . . . 9

6 Multi-Solver Fallback Architecture **9**
6.1 Algorithm Description . . . 9
6.1.1 Best-Point Tracking . . . 10
6.1.2 Dual Stagnation Detection . . . 10
6.2 Source Code References . . . 10

7 Adaptive Barrier Parameter **10**
7.1 Algorithm Description . . . 10
7.1.1 Barrier Oracle with Mehrotra Integration . . . 10
7.1.2 Mehrotra Predictor-Corrector and Gondzio Corrections . . . 11
7.1.3 Free and Fixed Modes . . . 11
7.1.4 Fraction-to-Boundary . . . 11
7.1.5 No-Bounds Special Case . . . 11
7.2 Source Code References . . . 11

8 Filter Line Search **11**
8.1 Algorithm Description . . . 11
8.1.1 Switching Condition . . . 12
8.1.2 Second-Order Correction (SOC) . . . 12
8.1.3 Objective Safeguard . . . 12
8.2 Source Code References . . . 12

9 Linear Algebra **12**
9.1 Algorithm Description . . . 12
9.1.1 Sparse Solver Dispatch . . . 12
9.1.2 rmumps: Pure Rust Multifrontal LDL^T . . . 13
9.1.3 Bunch-Kaufman LDL^T Factorization (Dense) . . . 13
9.1.4 Inertia Correction . . . 13
9.1.5 Iterative Refinement . . . 13
9.2 Source Code References . . . 14
9.2.1 rmumps Integration . . . 14
9.2.2 Dense Solver . . . 14
9.2.3 Inertia Correction and Search Direction . . . 14

10 Implementation **14**
10.1 Problem Interface . . . 14
10.2 Public API . . . 15
10.3 AMPL NL Interface and Pyomo Integration . . . 15
10.4 Julia/JuMP Interface (Ripopt.jl) . . . 15

11 Parametric Sensitivity Analysis **16**
11.1 Algorithm Description . . . 16
11.2 Source Code References . . . 16

12 Running the Benchmarks **17**
12.1 Prerequisites . . . 17
12.2 Hock-Schittkowski (HS) Suite . . . 17
12.2.1 Generating HS Problem Code . . . 17
12.2.2 Running ripopt on HS Suite . . . 17
12.2.3 Running Ipopt (native) on HS Suite . . . 17
12.2.4 Running cyipopt on HS Suite . . . 18
12.2.5 Comparing Results . . . 18
12.3 CUTEst Benchmark Suite . . . 18
12.3.1 Prerequisites . . . 18
12.3.2 Running CUTEst Benchmarks . . . 18
12.3.3 Interpreting Results . . . 18
12.4 Unit Tests . . . 19
12.5 Pyomo Integration . . . 19
12.6 Julia/JuMP Integration . . . 19

Contents

1 Introduction

This Supporting Information provides algorithmic details and source code references for each section of the manuscript. Source code locations are given as org-mode links in the format `[[file:line]]` that point to the ripopt repository root at `src/`. All line numbers refer to the current version of the source code at the time of writing.

The ripopt source code is organized as follows:

File	Purpose
`src/lib.rs`	Public API (`solve`, `solve_with_sensitivity`)
`src/problem.rs`	`NlpProblem` trait definition
`src/options.rs`	`SolverOptions` struct
`src/result.rs`	`SolveResult` and `SolveStatus` types
`src/ipm.rs`	Core IPM engine, NE-to-LS, fallback logic
`src/kkt.rs`	KKT assembly, condensed KKT, inertia correction
`src/convergence.rs`	Convergence checks
`src/filter.rs`	Filter line search and fraction-to-boundary
`src/restoration.rs`	Gauss-Newton feasibility solver (Phase 1)
`src/restoration_nlp.rs`	NLP restoration subproblem (Phase 2)
`src/slack_formulation.rs`	Explicit slack reformulation fallback
`src/lbfgs.rs`	L-BFGS solver
`src/augmented_lagrangian.rs`	Augmented Lagrangian solver
`src/warmstart.rs`	Warm-starting utilities
`src/sensitivity.rs`	Parametric sensitivity analysis (sIPOPT-style)
`src/linear_solver/mod.rs`	`SymmetricMatrix`, `LinearSolver` trait, matvec
`src/linear_solver/dense.rs`	Dense Bunch-Kaufman $\mathrm{LDL}^{\mathrm{T}}$ factorization
`src/linear_solver/multifrontal.rs`	rmumps multifrontal sparse $\mathrm{LDL}^{\mathrm{T}}$ (default)
`src/linear_solver/sparse.rs`	faer sparse $\mathrm{LDL}^{\mathrm{T}}$ (optional, `feature`"faer"=)
`src/linear_solver/banded.rs`	Banded $\mathrm{LDL}^{\mathrm{T}}$ factorization
`src/c_api.rs`	C API bindings for external language integration
`src/nl/parser.rs`	AMPL NL file parser
`src/nl/problem_impl.rs`	`NlProblem` (NlpProblem for NL files)
`src/nl/autodiff.rs`	Automatic differentiation for NL expressions
`src/bin/ripopt_ampl.rs`	AMPL binary interface
`gams/gams_ripopt.c`	GAMS solver link (GMO API to ripopt C API)

2 Implicit Slack Formulation

2.1 Algorithm Description

The implicit slack formulation eliminates explicit slack variables for inequality constraints. Instead of introducing variables s and enforcing $g(x) - s = 0$, $g_L \leq s \leq g_U$, the algorithm

treats the constraint slacks $s_L = g(x) - g_L$ and $s_U = g_U - g(x)$ as implicit functions of x.

For each inequality constraint i, the algorithm computes a barrier stiffness $\Sigma_{s,i}$ from the implicit slack multipliers, which are derived from the Lagrange multiplier y_i. When $y_i < 0$ (for a lower-bounded constraint), $z_{s_L,i} = -y_i$; otherwise, $z_{s_L,i} = \mu/\tilde{s}_{L,i}$ where $\tilde{s}_{L,i}$ is a safeguarded slack.

The barrier stiffness enters the $(2,2)$ block of the KKT matrix as $-\Sigma_s^{-1}$, producing a system of dimension $(n+m)$ instead of $(n+m+n_{\text{ineq}})$.

When a constraint is infeasible ($s_{L,i} < 0$), no barrier term exists and the constraint receives a pure feasibility-driving right-hand side.

2.2 Source Code References

- KKT assembly function: src/kkt.rs:52 (`assemble_kkt`)
- Σ_s computation and (2,2) block: src/kkt.rs:162 (=let mut sigma$_\text{s}$ = 0.0=)
- (2,2) block entry written: src/kkt.rs:209 (`matrix.add(n + i, n + i, -sigma_s_inv)`)
- Mathematical derivation comment: src/kkt.rs:106

2.3 Condensed KKT via Schur Complement

When $m \geq 2n$, the KKT system is reduced via Schur complement elimination of Δy. The condensed system $S\Delta x = r$ has dimension n instead of $n+m$, where $S = H + \Sigma_x + \delta_w I + J^T(-D_c)^{-1}J$.

For equality constraints with zero $(2,2)$ entry, a stiff-spring penalty of -10^{-16} prevents dropping them from the Schur complement.

- `CondensedKktSystem` struct: src/kkt.rs:1154
- `assemble_condensed_kkt` function: src/kkt.rs:1180
- Schur complement assembly ($S = H + \Sigma + J^T D_c^{-1} J$): src/kkt.rs:1289
- Threshold ($m \geq 2n$) in solver: src/ipm.rs:2852

3 Convergence Test

3.1 Algorithm Description

ripopt uses the same convergence test as Ipopt 3.14.x: two independent checks on the iterative multipliers, both of which must pass at the current iterate for `Optimal` status. Earlier ripopt releases computed auxiliary stationarity-based bound multipliers z_{opt} and used them in a separate scaled gate; v0.7.0 dropped that machinery after confirming that the iterative z_L, z_U (with fraction-to-boundary updates and Ipopt's κ_σ safeguarding) are sufficient once the barrier subproblem has been driven to a small μ.

3.1.1 Dual Scaling s_d

Following Ipopt, the scaled tolerances for the dual-infeasibility and complementarity gates are divided by

$$s_d = \max\left(s_{\max},\ \frac{\|y\|_1 + \|z_L\|_1 + \|z_U\|_1}{n+m}\right) / s_{\max},$$

with $s_{\max} = 100$ and s_d capped at 10^4 to prevent exploding multipliers from masking a non-converged iterate.

3.1.2 Two-Gate Test

Convergence at the user tolerance ϵ requires **both**:

1. **Scaled gate**: $E_\mu / s_d \leq \epsilon$ for dual infeasibility and complementarity; primal infeasibility against ϵ directly.
2. **Unscaled gate**: primal infeasibility $\leq$ `constr_viol_tol`, dual infeasibility $\leq$ `dual_inf_tol`, complementarity $\leq$ `compl_inf_tol`.

All three residuals use the iterative z values (matching Ipopt's `curr_dual_infeasibility`). An **acceptable** level mirroring Ipopt's defaults (`acceptable_tol` $= 10^{-6}$, 15 consecutive iterations, relaxed unscaled tolerances of 10^{-2}) is also tracked, but ripopt returns `Optimal` only when the strict two-gate test passes at the user tolerance.

3.2 Source Code References

- `check_convergence` function: src/convergence.rs:54
- `ConvergenceInfo` struct: src/convergence.rs:23
- Dual scaling s_d: src/convergence.rs:66
- Scaled gate: src/convergence.rs:83
- Unscaled gate: src/convergence.rs:86
- Acceptable-level defaults (internal, for best-du tracking): src/convergence.rs:101

4 Two-Phase Restoration with Multi-Attempt Recovery

4.1 Algorithm Description

4.1.1 Phase 1: Gauss-Newton Feasibility Solver

The GN phase minimizes $\frac{1}{2}\|h(x)\|^2$ where $h_i(x) = \max(g_{L,i} - g_i, g_i - g_{U,i}, 0)$. Steps:

1. Restrict the Jacobian to violated constraints: J_a.
2. Solve $(J_a J_a^T + \epsilon I)w = h_a$ via Levenberg-Marquardt regularization.
3. Compute the step $\Delta x = -J_a^T w$.
4. Backtracking line search on $\|h\|^2$.

Fallback mechanisms:

- **Gradient descent**: When GN line search fails, use $\Delta x = -J^T h$ with separate backtracking.
- **Proximity regularization**: Add pull toward starting point when progress stalls.
- **Penalty-regularized gradient descent**: Minimize $\frac{1}{2}\|h\|^2 + \rho\|x - x_{\text{ref}}\|^2$.

4.1.2 Phase 2: NLP Restoration

If GN fails on two consecutive occasions, the full NLP restoration subproblem is solved:

$$\min_{x,p,n} \rho \sum (p_i + n_i) + \frac{\eta}{2}\|D_R(x - x_R)\|^2, \quad g(x) - p + n = g_{\text{target}}, \quad p, n \geq 0$$

The subproblem is solved by the same IPM engine with =disable$_{\text{nlprestoration}}$=true= to prevent recursion. Dynamic dispatch (`&dyn NlpProblem`) breaks the monomorphization chain.

4.1.3 Multi-Attempt Recovery

When both phases fail, 6 recovery attempts are tried with alternating μ factors $[10, 0.1, 100, 0.01, 1000, 0.001]$ and deterministic x perturbations starting at attempt 3. After all attempts, infeasibility is checked via $\|J^T h(x)\|_\infty < 10^{-4} \cdot \max(\theta, 1)$.

4.1.4 Post-Restoration State Reset

After successful restoration: $z_i = \mu/s_i$ (capped at 10^3), $y_i = 0$, barrier mode reset to Free, filter cleared.

4.2 Source Code References

- `RestorationPhase` struct: src/restoration.rs:13
- Main GN loop (`restore` function): src/restoration.rs:42
- Gauss-Newton step computation: src/restoration.rs:439 (`gauss_newton_step`)
- Normal equations $J_a J_a^T$: src/restoration.rs:460
- Gradient descent fallback: src/restoration.rs:229

- `RestorationNlp` struct: src/restoration_nlp.rs:18
- `RestorationNlp::new`: src/restoration_nlp.rs:47
- Objective implementation ($\rho\Sigma + \eta/2\|D_R\|^2$): src/restoration_nlp.rs:200
- NLP restoration invocation: src/ipm.rs:4223 (`attempt_nlp_restoration`)
- NLP restoration triggered at $\text{fail}_{\text{count}}$==2: src/ipm.rs:4214
- Multi-attempt recovery loop: src/ipm.rs:4300
- μ factors cycle: src/ipm.rs:4314 (`[10.0, 0.1, 100.0, 0.01, 1000.0, 0.001]`)
- Mode switch (attempt 1): src/ipm.rs:4317
- x perturbation (attempts 3+): src/ipm.rs:4326
- Post-restoration state reset: src/ipm.rs:8509 (`apply_restoration_success`)
- Multiplier reset: src/ipm.rs:8527
- Filter reset: src/ipm.rs:8533
- μ recomputed: src/ipm.rs:8537
- Mode reset to Free: src/ipm.rs:8544

5 Automatic NE-to-LS Reformulation

5.1 Algorithm Description

5.1.1 Detection

At the initial point, five conditions are checked:

1. $m \geq n$
2. $|f(x_0)| < 10^{-10}$
3. $\|\nabla f(x_0)\|_\infty < 10^{-10}$
4. $g_{L,i} = g_{U,i}$ for all i (all equalities)
5. $\theta(x_0) \geq 10^{-8}$ (constraints not already satisfied)

5.1.2 Reformulation

The NE problem is reformulated as unconstrained (or bound-constrained) least squares:

$$\min_x F(x) = \frac{1}{2}\sum_i (g_i(x) - t_i)^2$$

5.1.3 Full Hessian

The Hessian includes both the Gauss-Newton approximation and the second-order correction:

$$\nabla^2 F(x) = J^T J + \sum_i r_i \nabla^2 g_i$$

The second-order correction reuses the NLP Hessian interface: calling `hessian_values(x, 0, r)` with $\sigma = 0$ and $\lambda = r$ directly computes $\sum_i r_i \nabla^2 g_i$.

5.1.4 Fallback

For square systems ($m = n$), if the LS reformulation returns `LocalInfeasibility` (nonzero residual minimum), the algorithm retries with the constrained IPM. The LS iteration count is capped at $\min(100, \text{max_iter}/10)$ for square systems.

5.2 Source Code References

- NE detection function: src/ipm.rs:1223 (`detect_ne_problem`)
- Detection trigger in `solve`: src/ipm.rs:1985
- `LeastSquaresProblem` struct: src/ipm.rs:980
- Full Hessian (`hessian_values`): src/ipm.rs:1097
- $J^T J$ (Gauss-Newton term): src/ipm.rs:1118
- $\sum r_i \nabla^2 g_i$ (second-order correction): src/ipm.rs:1129

6 Multi-Solver Fallback Architecture

6.1 Algorithm Description

When the primary IPM fails (`RestorationFailed`, `MaxIterations`, or `NumericalError`), up to three alternative solvers are attempted:

1. **L-BFGS**: For unconstrained problems, tried before IPM (no barrier needed). For bound-constrained, tried as fallback with projected gradient. Uses $k = 10$ curvature pairs, Wolfe line search.
2. **Augmented Lagrangian (AL)**: For constrained problems. PHR formulation with penalty parameter ρ increasing by $10\times$ when violation fails to decrease. Each AL subproblem is solved by the IPM engine.
3. **Explicit Slack Reformulation**: For inequality-constrained problems. Converts $g_L \leq g(x) \leq g_U$ to $g(x) - s = 0$, $g_L \leq s \leq g_U$, solved by IPM with =enable$_{\text{slackfallback}}$=false=. Triggered on any non-Optimal result for strict objective comparison.

6.1.1 Best-Point Tracking

Throughout the solve, the iterate with lowest dual infeasibility is tracked. At max iterations, this point is restored and checked against acceptable convergence criteria with relaxed primal tolerance.

6.1.2 Dual Stagnation Detection

If no dual infeasibility improvement occurs for 500+ iterations, the best-du point is restored and checked for acceptable convergence.

6.2 Source Code References

- L-BFGS solver entry: src/lbfgs.rs:24 (`pub fn solve`)
- L-BFGS memory size: src/lbfgs.rs:13 (=$\text{LBFGS}_\text{M} = 10$=)
- AL solver entry: src/augmented_lagrangian.rs:198 (`pub fn solve`)
- `SlackFormulation` struct: src/slack_formulation.rs:17
- `SlackFormulation::new`: src/slack_formulation.rs:34
- Slack `NlpProblem` impl: src/slack_formulation.rs:74
- Slack fallback trigger: src/ipm.rs:1474
- Best-du initialization: src/ipm.rs:7481
- Best-du update: src/ipm.rs:7645
- Best-du restore at $\max_{\text{iter}}$: src/ipm.rs:6468
- Dual stagnation detection: src/ipm.rs:6749
- Stagnation threshold (500 iters): src/ipm.rs:6775
- Stagnation restore: src/ipm.rs:6790

7 Adaptive Barrier Parameter

7.1 Algorithm Description

7.1.1 Barrier Oracle with Mehrotra Integration

The barrier parameter update combines a Loqo-style oracle with Mehrotra predictor-corrector feedback. In Free mode, the base update is $\mu = \bar{c}/\kappa$ where $\bar{c}$ is the average complementarity and $\kappa = 10$. When Mehrotra predictor-corrector is enabled (default), the centering parameter $\sigma = (\mu_{\text{aff}}/\mu)^3$ from the affine scaling probe is stored and used in the next mu update via a geometric blend: $\mu = \mu_\sigma^{1-\sigma} \cdot \mu_{\text{Loqo}}^{\sigma}$ where $\mu_\sigma = \sigma \cdot \mu_{\text{current}}$. This allows aggressive mu decreases when the affine step is near full-length ($\sigma \to 0$) while staying conservative otherwise.

7.1.2 Mehrotra Predictor-Corrector and Gondzio Corrections

Enabled by default (=mehrotra$_{\text{pc}}$=true=, =gondzio$_{\text{mccmax}}$=3=). After factoring the KKT system, an affine predictor step ($\mu = 0$) probes the Newton direction. The centering parameter σ determines how much to reduce μ in the corrector RHS. Up to 3 Gondzio centrality corrections further improve centering by driving outlier complementarity pairs back toward the central path.

7.1.3 Free and Fixed Modes

- **Free mode**: μ updated via oracle (with Mehrotra blend) each iteration; filter reset each iteration.
- **Fixed mode**: μ decreases monotonically via $\mu_{\text{new}} = \min(\kappa_\mu \mu, \mu^{\theta_\mu})$ with $\kappa_\mu = 0.2$, $\theta_\mu = 1.5$. Subproblem convergence checked via $\epsilon_{\text{barrier}} \cdot \mu$.
- Mode switching based on sliding window of 4 KKT error values (=refs$_{\text{redfact}} = 0.999$=).

7.1.4 Fraction-to-Boundary

Free mode uses $\tau = \max(\tau_{\min}, 1 - e_{\text{NLP}})$ (more aggressive than Ipopt's $1 - \mu$).

7.1.5 No-Bounds Special Case

When no variable has finite bounds, $\mu = \mu_{\min} = 10^{-11}$ immediately.

7.2 Source Code References

- `MuMode` enum: src/ipm.rs:464
- `MuState` struct: src/ipm.rs:472
- Main barrier update block: src/ipm.rs:5282
- Free mode oracle ($\bar{c}/\kappa$): src/ipm.rs:5432
- Fixed mode decrease: src/ipm.rs:5507

8 Filter Line Search

8.1 Algorithm Description

The filter line search follows Fletcher & Leyffer (2002) as adapted by Wachter & Biegler (2006).

8.1.1 Switching Condition

The switching condition $\alpha(-\nabla\phi^T d)^{s_\phi} > \delta\theta^{s_\theta}$ includes the step size α as a multiplier. As α decreases during backtracking, the switching condition eventually fails, allowing h-type acceptance.

8.1.2 Second-Order Correction (SOC)

SOC is attempted on every backtracking step where θ increases (not just the first). Up to $k_{\text{soc}} = 4$ SOC iterations are performed with convergence check $\theta_{\text{soc}} \geq \kappa_{\text{soc}}\theta_{\text{prev}}$ ($\kappa_{\text{soc}} = 0.99$).

8.1.3 Objective Safeguard

Reject trial points where $\phi_{\text{trial}} > \phi_{\text{current}} + 5(1 + |\phi_{\text{current}}|)$.

8.2 Source Code References

- `Filter` struct: src/filter.rs:14
- `is_acceptable`: src/filter.rs:64
- `switching_condition`: src/filter.rs:86
- `armijo_condition`: src/filter.rs:102
- `check_acceptability`: src/filter.rs:133
- `fraction_to_boundary`: src/filter.rs:270
- Full KKT SOC: src/ipm.rs:8310 (`attempt_soc`)
- Condensed SOC: src/ipm.rs:8349 (`attempt_soc_condensed`)
- SOC trigger on backtracking: src/ipm.rs:2815
- κ_{soc} check (full): src/ipm.rs:8197
- κ_{soc} check (condensed): src/ipm.rs:8197

9 Linear Algebra

9.1 Algorithm Description

9.1.1 Sparse Solver Dispatch

The IPM automatically selects the linear solver based on problem size and available features. For systems with dimension < 110 (configurable via `sparse_threshold`), a dense solver is used. For larger systems, the default sparse solver is `rmumps` (a pure Rust multifrontal solver).

For tall-narrow problems ($m \geq 2n$ with $n \leq 100$), the dense condensed KKT path is used even when $n + m$ exceeds the sparse threshold, replacing an $(n + m) \times (n + m)$ sparse factorization with an $n \times n$ dense solve. This gives 100–800× speedup on problems like EXPFITC ($n = 5$, $m = 502$) and OET3 ($n = 4$, $m = 1002$).

9.1.2 rmumps: Pure Rust Multifrontal LDL$^{\mathbf{T}}$

The default sparse solver is `rmumps`, a pure Rust multifrontal solver (CeCILL-C license) included as a workspace member. It implements:

1. **SuiteSparse AMD fill-reducing ordering** (via the `amd` crate) to minimize fill during factorization.
2. **Supernodal symbolic factorization** to identify groups of columns with the same sparsity structure.
3. **Level-set parallel numeric factorization**: supernodes at the same depth in the elimination tree are independent and factored concurrently via `rayon`.
4. **Dense frontal operations**: each supernode does dense BK pivoting on its fully-summed block, computes the L_{21} sub-diagonal factor, and forms the Schur complement $S = A_{22} - L_{21} D L_{21}^T$ via SIMD-accelerated GEMM (NEON on aarch64, AVX2+FMA on x86$_{64}$).
5. **Iterative refinement**: 2 rounds by default to improve solution accuracy.

The `MultifrontalLdl` wrapper caches the COO-to-CSC mapping and symbolic analysis across calls with the same sparsity pattern, so that repeated factorizations during inertia correction only perform numeric refactorization.

9.1.3 Bunch-Kaufman LDL$^{\mathbf{T}}$ Factorization (Dense)

For systems with dimension < 110, a dense LDL^{T} factorization with Bunch-Kaufman pivoting is used. This produces $PLDL^T P^T = A$ where D has 1×1 and 2×2 blocks. The 2×2 blocks handle the indefinite structure of the KKT matrix.

A critical implementation detail: when rows/columns are swapped during pivoting, L entries from previously computed columns must also be swapped.

9.1.4 Inertia Correction

The KKT matrix must have inertia $(n, m, 0)$. After factorization, inertia is extracted from D and regularization δ_w (to $(1, 1)$ block) and $-\delta_c$ (to $(2, 2)$ block) is added if incorrect. The regularization grows exponentially ($\delta_w \to 4\delta_w$) for up to 15 attempts. On failure after all attempts, the algorithm proceeds with the approximate factorization.

9.1.5 Iterative Refinement

Up to 3 rounds of iterative refinement with tolerance 10^{-12}.

9.2 Source Code References

9.2.1 rmumps Integration

- `MultifrontalLdl` struct: src/linear_solver/multifrontal.rs:9
- `LinearSolver` impl for `MultifrontalLdl`: src/linear_solver/multifrontal.rs:115
- COO-to-CSC mapping cache: src/linear_solver/multifrontal.rs:78 (`build_coo_to_csc_mapping`)
- Scatter COO values to cached CSC: src/linear_solver/multifrontal.rs:101 (`scatter_coo_to_csc`)
- `LinearSolver` trait: src/linear_solver/mod.rs:567
- `KktMatrix` enum (Dense/Sparse): src/linear_solver/mod.rs:467

9.2.2 Dense Solver

- `DenseLdl` struct: src/linear_solver/dense.rs:11
- `bunch_kaufman_factor`: src/linear_solver/dense.rs:65
- `find_pivot`: src/linear_solver/dense.rs:225
- `compute_inertia`: src/linear_solver/dense.rs:299
- `solve_internal`: src/linear_solver/dense.rs:351

9.2.3 Inertia Correction and Search Direction

- `InertiaCorrectionParams` struct: src/kkt.rs:344
- `factor_with_inertia_correction`: src/kkt.rs:453
- Perturbation loop: src/kkt.rs:610
- Proceed-on-failure logic: src/kkt.rs:680
- `solve_for_direction` (iterative refinement): src/kkt.rs:737
- Refinement loop: src/kkt.rs:779

10 Implementation

10.1 Problem Interface

The `NlpProblem` trait defines a buffer-filling interface for optimization problems. All evaluation methods take pre-allocated slices to avoid heap allocation in the hot loop.

- `NlpProblem` trait: src/problem.rs:17

10.2 Public API

The public API consists of two functions:

- `solve<P: NlpProblem>(problem: &P, options: &SolverOptions) -> SolveResult` for standard optimization.
- `solve_with_sensitivity<P: ParametricNlpProblem>(problem: &P, options: &SolverOpti -> (SolveResult, SensitivityContext)` for optimization with parametric sensitivity analysis.
- `solve` function: src/lib.rs:107
- `solve_with_sensitivity` function: src/lib.rs:112
- Internal solver entry: src/ipm.rs:2076 (dispatches NE-to-LS, fallbacks, then `solve_ipm`)
- `solve_ipm` (core IPM loop): src/ipm.rs:7359

10.3 AMPL NL Interface and Pyomo Integration

The AMPL interface reads NL files (nonlinear expression DAGs) with automatic differentiation for gradient, Jacobian, and Hessian evaluation, and writes SOL files.

- NL file parser: src/nl/parser.rs:48 (`parse_nl_file`)
- `NlFileData` struct: src/nl/parser.rs:18
- `NlProblem` (NlpProblem impl): src/nl/problem_impl.rs:11
- `NlpProblem for NlProblem` impl: src/nl/problem_impl.rs:240
- AMPL binary (reads `.nl`, writes `.sol`): src/bin/ripopt_ampl.rs
- Pyomo plugin: `pyomo-ripopt/` directory

10.4 Julia/JuMP Interface (Ripopt.jl)

`Ripopt.jl` wraps ripopt's C API behind a MathOptInterface (MOI) `Optimizer` type, enabling use from JuMP. The wrapper implements the incremental MOI interface so that `MOI.Utilities.default_copy_to` can populate the optimizer from JuMP's internal cache.

Key source files:

- Module entry point and `__init__` (runtime library path): Ripopt.jl/src/Ripopt.jl
- C API bindings (`RipoptProblem`, `CreateRipoptProblem`, `RipoptSolve`, options, statistics): Ripopt.jl/src/C_wrapper.jl
- MOI wrapper (`Optimizer` struct, `copy_to`, `optimize!`, result accessors): Ripopt.jl/src/MOI_wrapper.jl

Design notes:

- The five C callbacks (`eval_f`, `eval_grad_f`, `eval_g`, `eval_jac_g`, `eval_h`) are module-level functions, not closures. This is required on ARM/AArch64 (Apple Silicon) where Julia does not support trampoline-backed `@cfunction` closures (the `$f` form). All problem data is passed through the `user_data` pointer as a GC-protected `Ref{NamedTuple}`.
- The library path is a mutable global set in `__init__`, not a `const`. This ensures the `RIPOPT_LIBRARY_PATH` environment variable is read at module load time, so the precompiled module image is not tied to a specific build directory.
- `MOI.supports_incremental_interface` returns `true`; `copy_to` delegates to `MOI.Utilities.defaul` Variable bounds go through `MOI.VariablesContainer`; the nonlinear objective and constraints go through `MOI.NLPBlock`.
- Option dispatch checks `Integer` before `Real` to avoid routing integer-valued options (e.g., `max_iter`) to the floating-point option setter.

11 Parametric Sensitivity Analysis

11.1 Algorithm Description

The sensitivity analysis implements the sIPOPT approach: after the IPM converges, the KKT system is reassembled *without* inertia correction ($\delta_w = 0$, $\delta_c = 0$) and factored. For each parameter perturbation Δp, the RHS is assembled from user-provided parameter derivatives ($\nabla^2_{xp} L$ and $\partial g/\partial p$), and the sensitivity is obtained by a single backsolve.

The key design decision is to *not* retain the KKT matrix from the final IPM iteration. The final KKT may include inertia correction terms that would introduce $O(\delta)$ bias. Instead, we pay the cost of one extra factorization for exact sensitivities.

Bound multiplier sensitivities are derived from complementarity. From $z_L(x - x_L) = \mu$ at the barrier solution, differentiating with respect to p gives $dz_L = -z_L\, dx/(x - x_L)$. Similarly for upper bounds: $dz_U = z_U\, dx/(x_U - x)$.

The reduced Hessian is extracted column-by-column from M^{-1} by solving $Me_j = \mathrm{col}_j$ for $j = 1, \ldots, n$. The $n \times n$ upper-left block gives the projected Hessian inverse, useful for covariance estimation in parameter estimation problems.

11.2 Source Code References

- `ParametricNlpProblem` trait: src/sensitivity.rs:19
- `SensitivityContext` struct: src/sensitivity.rs:39
- `solve_with_sensitivity()` function: src/sensitivity.rs:72
- KKT reassembly (unregularized): src/sensitivity.rs:121
- `compute_sensitivity()` backsolve: src/sensitivity.rs:161

- `reduced_hessian()` extraction: src/sensitivity.rs:286
- Unit tests (parametric QP, constrained QP, reduced Hessian): src/sensitivity.rs:325
- Integration tests (HS071 parametric, finite difference verification): tests/sensitivity.rs:1
- Example (HS071 with parametric constraint bound): examples/sensitivity.rs:1

12 Running the Benchmarks

12.1 Prerequisites

- Rust toolchain (stable): https://rustup.rs/
- For CUTEst benchmarks: CUTEst library and Ipopt (with MUMPS) installed
- For HS comparisons with cyipopt: Python 3 with `cyipopt` package

12.2 Hock-Schittkowski (HS) Suite

12.2.1 Generating HS Problem Code

The HS problems are generated from Fortran source files:

```
cd benchmarks/hs
python generate.py   # generates benchmarks/hs/generated/hs_problems.rs
```

12.2.2 Running ripopt on HS Suite

```
cargo run --release --features hs --bin hs_suite
```

This solves the HS problems and outputs JSON results to stdout. The number of timing runs can be controlled via environment variable:

```
RIPOPT_TIMING_RUNS=10 cargo run --release --features hs --bin hs_suite \
    > benchmarks/hs/hs_ripopt_results.json
```

To force sparse factorization on all problems (for comparison):

```
RIPOPT_FORCE_SPARSE=1 cargo run --release --features hs --bin hs_suite
```

12.2.3 Running Ipopt (native) on HS Suite

Requires the `ipopt-native` feature and Ipopt C library:

```
cargo run --release --features ipopt-native --bin ipopt_native \
    > benchmarks/hs/hs_ipopt_native_results.json
```

12.2.4 Running cyipopt on HS Suite

```
cd benchmarks/hs
python run_cyipopt.py > hs_cyipopt_results.json
```

12.2.5 Comparing Results

```
cd benchmarks/hs
python compare.py
```

This reads the JSON results and generates `HS_VALIDATION_REPORT.md` with solve rates, objective comparisons, speed comparisons, and iteration counts. A second, more detailed report is written to `benchmarks/hs/PERFORMANCE_COMPARISON.md`.

12.3 CUTEst Benchmark Suite

12.3.1 Prerequisites

CUTEst must be installed and the environment configured so that the CUTEst shared libraries and SIF files are available. The `CUTEST` and `MASTSIF` environment variables should point to the CUTEst installation and SIF problem collection respectively.

12.3.2 Running CUTEst Benchmarks

To run specific problems:

```
cargo run --release --features cutest,ipopt-native --bin cutest_suite -- ROSENBR HS35 HS71
```

To run all problems from the problem list:

```
RESULTS_FILE=benchmarks/cutest/results.json \
    cargo run --release --features cutest,ipopt-native --bin cutest_suite
```

This reads problem names from `benchmarks/cutest/problem_list.txt` and solves each with both ripopt and native Ipopt. Results are auto-saved to `benchmarks/cutest/results.json` (override with `RESULTS_FILE`).

The solver uses =$\max_{\text{walltime}}$=30s= so the IPM exits before the 60-second OS kill limit, enabling fallback solvers to attempt recovery.

12.3.3 Interpreting Results

Results are saved in JSON format with fields:

- `name`: CUTEst problem name
- `solver`: "ripopt" or "ipopt"

- `n`, `m`: problem dimensions
- `status`: solver status string
- `objective`: final objective value
- `constraint_violation`: final constraint violation
- `iterations`: number of iterations
- `solve_time`: wall-clock time in seconds

12.4 Unit Tests

The correctness test suite covers fundamental problem types:

```
cargo test --release
```

Test problems include Rosenbrock, simple QP, HS071, bound-constrained QP, multi-equality, and pure bound problems. See `tests/correctness.rs` for details.

12.5 Pyomo Integration

To test the Pyomo integration:

```
# Build the ripopt AMPL binary
cargo build --release --bin ripopt

# Install the Pyomo plugin
cd pyomo-ripopt
pip install -e .

# Test
python -c "
import pyomo_ripopt
from pyomo.environ import *
model = ConcreteModel()
model.x = Var(initialize=1.5, bounds=(1, 5))
model.y = Var(initialize=1.5, bounds=(1, 5))
model.obj = Objective(expr=(model.x - 1)**2 + (model.y - 2)**2)
model.c1 = Constraint(expr=model.x + model.y <= 4)
solver = SolverFactory('ripopt')
result = solver.solve(model, tee=True)
print(f'x={value(model.x):.6f}, y={value(model.y):.6f}, obj={value(model.obj):.6f}')
"
```

12.6 Julia/JuMP Integration

To run the Julia examples (requires Julia 1.9 with JuMP installed):

```
# Build the shared library
cargo build --release

# Run the JuMP examples
```

```
RIPOPT_LIBRARY_PATH=target/release julia --project=@v1.12 \
    Ripopt.jl/examples/jump_hs071.jl

RIPOPT_LIBRARY_PATH=target/release julia --project=@v1.12 \
    Ripopt.jl/examples/jump_rosenbrock.jl

# Low-level C wrapper example (no JuMP required)
RIPOPT_LIBRARY_PATH=target/release julia --project=@v1.12 \
    Ripopt.jl/examples/c_wrapper_hs071.jl
```

Expected output for `jump_hs071.jl`:

```
Termination status: LOCALLY_SOLVED
Objective value:    17.014017289919348
Solution:           [1.0, 4.743, 3.821, 1.379]
```

To install `Ripopt.jl` into a Julia environment:

```
import Pkg
Pkg.develop(path="Ripopt.jl")   # or Pkg.add(url="...") for remote
```

The interactive notebook tutorial is at `Ripopt.jl/examples/ripopt_jump_tutorial.ipynb`.